בס"ד

שפות תכנות – תרגיל 1

תאריך הגשה: 16.11.2025

הוראות הגשה: ההגשה בזוגות. כל זוג נדרש לחשוב, לפתור ולכתוב את התרגיל בעצמו. מותר להתייעץ עם זוגות אחרים אך חל איסור מוחלט להחזיק ולהיעזר בתרגיל כתוב של זוג אחר.
יש לקרוא הוראות אלא בקפידה, הגשה שלא על פי הוראות אלה תוביל להורדת ניקוד ולא יתקבלו על כך ערעורים!

יש להגיש אך ורק את התשובה ללא תוספות (כמו שורת פקודה להרצת הקובץ שראינו בתרגול).

חומר עזר מומלץ: כדאי להבין היטב את הרצאות ותרגולים מספר 1-2. קישור לתרגולים (המצגות נמצאות בתיאור הסרטון) מופיע בלמדה.

קישור להסבר על השערת גולבך:

https://davidson.weizmann.ac.il/online/mathcircle/articles/%D7%94%D7%A9%D7%A2%D7%A8%D7%AA-%D7%92%D7%95%D7%9C%D7%93%D7%91%D7%9A-%D7%90%D7%95-%D7%9E%D7%A9%D7%A4%D7%98%D7%99%D7%9D-%D7%A4%D7%A9%D7%95%D7%98%D7%99%D7%9D-%D7%A2%D7%9D-%D7%94%D7%95%D7%9B%D7%97%D7%94-%D7%9C%D7%90-%D7%A4%D7%A9%D7%95%D7%98%D7%94

סקריפט בדיקה: לחלק התכנות בפרולוג מצורף סקריפט בדיקה בשם test.sh שצריך להיות באותה תיקייה עם קטעי הקוד ועם הקובץ test.txt המצורף. כדי להריץ את הסקריפט יש להריץ את הפקודה:

```
sudo bash test.sh
```

שימו לב שהסקריפט שולח אזהרה – ניתן להתעלם ממנה. כמו כן הסקריפט בודק את הפלט שלכם עם כל קטעי הקוד ולכן יהיו שגיאות אם לא מימשתם עדיין את כל הסעיפים. שימו לב: ייתכנו מקרים בהם מספר זוגי יכול להיכתב כסכום של שני מספרים ראשוניים. למשל, המספר 24 יכול להכתב כסכום 11+13 אך גם כסכום 19+5. הסקריפט מחזיר רק אופציה אחת כזו וזה בסדר גמור אם התכנית שתממשו תחזיר אופציה אחרת .

ייתכן שתצליחו להריץ את הסקריפט ללא sudo. בנוסף הסקריפט מניח ש-swipl מותקן אצלכם במקום הדיפולטיבי בהינתן שלא צריך לשנות את הכתובת שלו בסקריפט.

אין חובה להריץ את הסקריפט אבל אני אבדוק עם סקריפט דומה ולכן מומלץ.

מה להגיש:

קבצי התכנות:

goldbach.pl, element_at.pl, table.pl, remove_at.pl, is_prime.pl

יש להגיש את כל הקבצים ביחד עם קובץ id.txt בקובץ zip בשם ex1.zip

- **חלק א: prolog**

בחלק זה נתכנת בשפת prolog, עבור כל סעיף צריך לכתוב קובץ pl. שהשם שלו הוא שם ה"פונקציה" שאותה יש לכתוב. לדוגמא עבור שאלה אחת צריך להגיש קובץ element_at.pl כך שניתן יהיה להפעיל את הפרדיקט לאחר ביצוע הפקודה .consult(element_at)

1) כתבו פרדיקט בשפת prolog בשם element_at שמקבל משתנה X, רשימה ומספר שלם k כלשהו ומחזיר את האיבר ה-k ברשימה. האיבר הראשון ברשימה הוא איבר מספר 1, ניתן להניח שלא יתקבל k הגדול מאורך הרשימה או קטן שווה לאפס.
דוגמא:

```
?- element_at(X,[a,b,c,d,e],3).
X = c
```

2) כתבו פרדיקט בשפת prolog בשם remove_at שמקבל משתנה X, רשימה ומספר שלם k כלשהו (עד כאן בדיוק כמו התרגיל הקודם) ובנוסף גם משתנה R. הפרדיקט יחזיר במשתנה X את האיבר ה-k (בדיוק כמו השאלה הקודמת) ובמשתנה R הפרדיקט יחזיר את הרשימה ללא האיבר ה-k (שנמצא ב-X).
דוגמא:

```
?- remove_at(X,[a,b,c,d],2,R).
X = b
R = [a,c,d]
```

3) כתבו פרדיקט בשפת prolog בשם is_prime שמקבל מספר שלם כלשהו ומחזיר אמת אם המספר הוא ראשוני ושקר אחרת.
דוגמא:

```
?- is_prime(7).
true

?- is_prime(15).
false
```

4) כתבו פרדיקט בשפת prolog בשם goldbach שמקבל מספר ומשתנה X ומחזיר ב-X רשימה המכילה שני מספרים ראשוניים שסכומם הוא המספר בארגומנט הראשון לפונקציה (במידה ולא קיימים החזירו רשימה ריקה).
דוגמא:

```
?- goldbach(28, X).
X = [5,23]
```

כאשר יש יותר מתשובה אחת ניתן להחזיר רק אחת.

5) הגדירו את הפרדיקטים הבאים בשפת prolog:
and, or, not, xor, nand, nor, equal
כל פרדיקט מקבל שני ארגומנטים (למעט הפרדיקט של not שמקבל ארגומנט אחד) ומחזיר אמת או שקר.
כעת כתבו פרדיקט table המקבל שני משתנים וביטוי בוליאני המשתמש בפרדיקטים שהגדרתם **וכותב למסך** טבלת אמת עבור הביטוי שהתקבל.
דוגמא:

```
?- table(A,B, and(A,(or(A,not(B))))).
true true true
true fail true
fail true fail
fail fail fail
```

ניתן להניח תקינות של הקלט.